Date:-8/10/2026

Week-2

Particle Swarm Optimization for Function Optimization:

Pseudocode

START

1. Define the objective function f(x)

2. Initialize PSO parameters:

   Number of particles (N)

   Maximum iterations (MAX)

   Inertia weight (w)

   Cognitive coefficient (c1)

   Social coefficient (c2)

3. Initialize particles:

   Generate random positions (X)

   Generate random velocities (V)

4. For each particle:

   Calculate fitness = f(X)

   Set personal best (Pbest) = X

5. Find the particle with the best fitness

   Set global best (Gbest) = best Pbest

6. Repeat until maximum iterations:

   For each particle:

      Update velocity:

V = w × V+ c1 × r1 × (Pbest - X) + c2 × r2 × (Gbest - X)

Update position:

X = X + V

Calculate new fitness = f(X)

If new fitness is better than Pbest fitness:

Update Pbest = X

If Pbest fitness is better than Gbest fitness:

Update Gbest = Pbest

7. Display:

Best position (Gbest)

Best fitness value f(Gbest)

STOP

Python Program: Particle Swarm Optimization (PSO)

```
import random

# Objective function
def fitness(x):
    return x ** 2

# PSO parameters
particles = 10
iterations = 50
```

```
w = 0.5
c1 = 1.5
c2 = 1.5

# Initialize particle positions and velocities
position = [random.uniform(-10, 10) for _ in range(particles)]
velocity = [random.uniform(-1, 1) for _ in range(particles)]

# Initialize personal best
pbest = position.copy()

# Initialize global best
gbest = min(pbest, key=fitness)

# PSO algorithm
for i in range(iterations):
    for j in range(particles):
        r1 = random.random()
        r2 = random.random()

        # Update velocity
        velocity[j] = (w * velocity[j]+ c1 * r1 * (pbest[j] - position[j]) + c2 * r2 * (gbest - position[j]))

        # Update position
        position[j] = position[j] + velocity[j]

        # Update personal best
        if fitness(position[j]) < fitness(pbest[j]):
            pbest[j] = position[j]
```

```
        # Update global best
        if fitness(pbest[j]) < fitness(gbest):
            gbest = pbest[j]

# Display result
print("Best Position:", gbest)
print("Minimum Fitness:", fitness(gbest))
```

⇨ **OUTPUT**

```
Best Position: -1.4525460212611166e-08
Minimum Fitness: 2.1098899438815003e-16
```